

浙江大学实验报告

课程名称: 数字系统设计实验 指导老师: 屈民军, 唐奕
实验名称: 实验 5 常用组合电路模块的设计和应用 实验类型: 基础实验

一、实验目的

- (1)掌握用 VerilogHDL 描述数据选择器、加法器和比较器等电路模块。
- (2)了解“自顶而下”的数字设计方法,掌握系统层次结构的设计。
- (3)掌握模块调用的方法,掌握参数定义和参数传递的方法。
- (4)掌握 ModelSim 功能仿真的工作流程,进一步了解 Vivado 的工作流程。
- (5)认识到文件管理的重要性。

二、实验任务

(1)设计两数之差的绝对值电路: 电路输入 aIn 、 bIn 为 4 位无符号二进制数, 电路输出 out 为两数之差的绝对值, 即 $out = |aIn - bIn|$ 。要求用多层次结构设计电路, 即调用数据选择器、加法器和比较器等基本模块来设计电路。

(2)设计模式比较器(Mode-Dependent Comparator)电路: 电路的输入为两个 8 位无符号二进制数 a 、 b 和一个模式控制信号 m ; 电路的输出为 8 位无符号二进制数 y 。当 $m=0$ 时, $y=MAX(a,b)$; 而当 $m=1$ 时, 则 $y=MIN(a,b)$ 。要求用多层次结构设计电路, 即调用数据选择器和比较器等基本模块来设计电路。

三、实验原理

1、两数之差的绝对值电路设计

两个无符号数之差的绝对值电路可由 (1) 式计算。另外, 减法可由补码加法完成, 因此 (1) 式可转为 (2) 式实现。

$$out = Max(aIn, bIn) - Min(aIn, bIn) \quad (1)$$

$$out = Max(aIn, bIn) + (\sim Min(aIn, bIn) + 1) \quad (2)$$

根据 (2) 式可画出两数之差的绝对值电路的顶层设计, 如图 1 所示。图中, 主要由三部分组成: 数值比较器(comp)实现比较输入两数的大小; 比较结果 agb 控制两个数据选择器, 数据选择器分别选出输入两数的最大值 Max 和最小值 Min ; 最后由全加器组成的四位加法器实现 Max 与 $(-Min)$ 的补码加法。

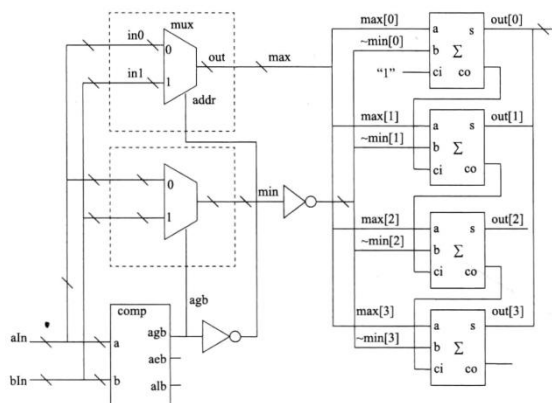


图 1 两数之差的绝对值电路的原理框图

2、模式比较器电路的顶层设计

根据实验要求可画出模式比较器电路的顶层设计，如图 2 所示，其中的组合电路产生数据选择的地址信号，其功能如表 1 所示，从功能表中可看出，addr 产生电路就是一个同或门。

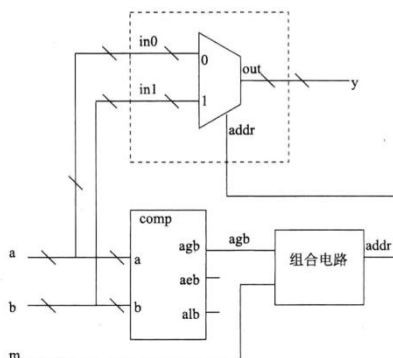


图 2 模式比较器电路的原理框图

m	agb	addr	备注
0	0	1	当 $m=0$ ，且 $a \leq b$ 时，y 取大值 b
0	1	0	当 $m=0$ ，且 $a > b$ 时，y 取大值 a
1	0	0	当 $m=1$ ，且 $a \leq b$ 时，y 取小值 a
1	1	1	当 $m=1$ ，且 $a > b$ 时，y 取小值 b

表 1 产生 addr 组合电路的功能表

四、实验设备

- (1)装有 Vivado、ModelSim SE 软件的计算机。
- (2)Nexys Video 开发板或 Basys3 开发板。

五、实验内容

1、任务一 两数之差的绝对值电路的设计

- (1)编写一位全加器的 Verilog HDL 代码，并用 ModelSim 软件进行功能仿真。
- (2)编写 n 位二选一数据选择器的 Verilog HDL 代码,并用 ModelSim 软件进行功能仿真。
- (3)编写 n 位比较器的 Verilog HDL 代码，并用 ModelSim 软件进行功能仿真。
- (4)对两数之差的绝对值电路进行功能仿真。
- (5)建立 Vivado 工程文件，对工程进行综合、引脚约束、实现，并下载到开发实验板中对设计进行验证。

2、任务二 模式比较器

编写模式比较器的 Verilog HDL 代码，并用 ModelSim 软件进行功能仿真。

六、实验代码

1、两数之差的绝对值电路的顶层模块

两数之差的绝对值电路代码由顶层设计模块 abs_dif 和功能模块数据比较器 comp、数据选择器 mux_2to1、一位全加器 full_adder 实现。

顶层设计模块的代码在书中已给出，这段代码首先调用数据比较器 comp 对输入的两个 4 位无符号二进制数 aln、bIn 比较大小，若 $aln > bIn$ ，则输出 $agb=1$ ，否则输出 $agb=0$ ；然后调用数据选择器 mux_2to1，根据 agb 的值得到最大值 max 和最小值 min，最后调用一位全加器 full_adder 实现最大值 max 和最小值 min 的相减，通过最大值 max 与最小值 min 的补码的相加来实现。

两数之差的绝对值电路代码顶层设计模块的具体代码如下：

```

module abs_dif(aIn,bIn,out);
  input [3:0] aIn,bIn;
  output[3:0] out;

  wire agb;
  comp #(.n(4)) comp_inst(.a(aIn),.b(bIn),.agb(agb),.aeb(),.alb());

  wire[3:0] max,min;
  mux_2to1 #(.n(4)) mux1(.out(max),.in0(aIn),.in1(bIn),.addr(~agb));
  mux_2to1 #(.n(4)) mux2(.out(min),.in0(aIn),.in1(bIn),.addr(agb));

  wire[2:0] c;
  full_adder adder0(.a(max[0]),.b(~min[0]),.s(out[0]),.ci(1'b1),.co(c[0]));
  full_adder adder1(.a(max[1]),.b(~min[1]),.s(out[1]),.ci(c[0]),.co(c[1]));
  full_adder adder2(.a(max[2]),.b(~min[2]),.s(out[2]),.ci(c[1]),.co(c[2]));
  full_adder adder3(.a(max[3]),.b(~min[3]),.s(out[3]),.ci(c[2]),.co());
endmodule

```

2、模式比较器电路的顶层模块

模式比较器电路代码由顶层设计模块 ModeComparator 和功能模块数据比较器 comp、数据选择器 mux_2to1 实现。

顶层设计模块的代码要求自行编写，这段代码首先调用数据比较器 comp 对输入的两个 8 位无符号二进制数 a、b 比较大小，若 $a > b$ ，则输出 $agb=1$ ，否则输出 $agb=0$ ；然后调用数据选择器 mux_2to1，根据 agb 的值选择输出 a、b 中的最大值或最小值。

模式比较器电路代码顶层设计模块的具体代码如下：

```

module ModeComparator(a,b,m,y);
  input [8:0] a, b;
  input m;
  output[8:0] y;

  wire agb;
  comp #(.n(8)) comp_inst(.a(a),.b(b),.agb(agb),.aeb(),.alb());

  mux_2to1 #(.n(8)) mux(.out(y),.in0(a),.in1(b),.addr(~(m^agb)));
endmodule

```

3、各功能模块的设计

(1) 数据选择器的设计

数据选择器的端口在书中已给出，我使用持续赋值语句和条件运算符实现选择功能，当 $addr=1$ 时，输出为 in1，反之则为 in0。

数据选择器的具体代码如下：

```

module mux_2to1(out,in0,in1,addr);
  parameter n=1;
  output[n-1:0] out;
  input[n-1:0] in0,in1;
  input addr;

```

```
assign out=addr?in1:in0;
endmodule
```

(2) 数据比较器的设计

数据比较器的端口在书中已给出，我使用 always 语句和 if 语句实现比较功能，根据 a 和 b 的大小关系决定 agb、aeb 和 alb 的值。

数据比较器的具体代码如下：

```
module comp(a,b,agb,aeb,alb);
parameter n=1;
input[n-1:0] a, b;
output reg agb;
output reg aeb;
output reg alb;

always @(a or b)
begin
if(a>b) {agb,aeb,alb}=3'b100;
else if(a==b) {agb,aeb,alb}=3'b010;
else {agb,aeb,alb}=3'b001;
end
endmodule
```

(3) 一位全加器的设计

一位全加器的端口在书中已给出，我使用数据流描述，依据全加器的逻辑表达式，采用持续赋值语句实现了全加功能。

一位全加器的具体代码如下：

```
module full_adder(a,b,s,ci,co);
input a,b,ci;
output s,co;

assign s=a^b^ci;
assign co=a&ci|b&ci|a&b;
endmodule
```

七、仿真测试

1、两数之差的绝对值电路

(1)一位全加器

一位全加器的 ModelSim 仿真波形如图 3 所示。在仿真波形中，低电平代表 0，高电平代表 1，如图，当 a=0, b=0, ci=1 时，s=1, co=0，以此类推，可以得出 a、b、ci 取不同值时，s、co 对应的值。将这些值与一位全加器的真值表（表 2）对照，发现两者完全一致，该模块程序编写正确，可以正常运行。

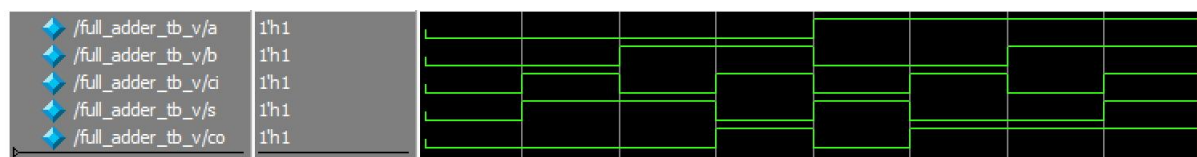


图 3 一位全加器的仿真波形

a	b	ci	s	co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

表 2 一位全加器的真值表

(2)数据选择器

数据选择器的 ModelSim 仿真波形如图 4 所示。如图，当 addr=0 时，输出为 in0；当 addr=1 时，输出的取值与该模块需要实现的功能一致，该模块程序编写正确，可以正常运行。

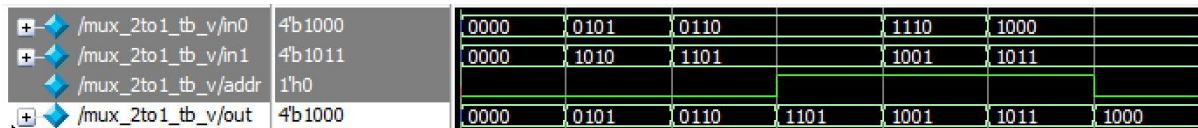


图 4 数据选择器的仿真波形

(3)四位比较器

四位比较器的 ModelSim 仿真波形如图 5 所示。如图，当 a=b 时，agb=0, aeb=1, alb=0；当 a<b 时，agb=0, aeb=0, alb=1；当 a>b 时，agb=1, aeb=0, alb=0，输出的取值与该模块需要实现的功能一致，该模块程序编写正确，可以正常运行。

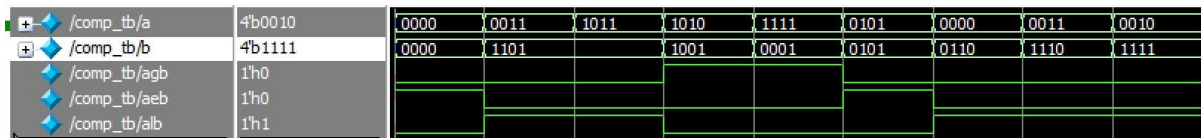


图 5 四位比较器的仿真波形

(4)两数之差的绝对值

两数之差的绝对值的 ModelSim 仿真波形如图 6 所示。如图，当 a=4'b0011, b=4'b1100 时，out=4'b1001=|a-b|，以此类推，可以得出 a 和 b 取不同值时，out 的值均是 a 和 b 差的绝对值，与该模块需要实现的功能一致，该模块程序编写正确，可以正常运行。

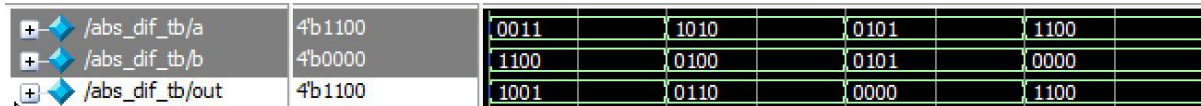


图 6 两数之差的绝对值的仿真波形

(5)建立 Vivado 工程文件

将任务一的所有程序导入 Vivado 软件中，对工程进行综合后得到的原理图如图 7 所示。将该原理图与图 1 的原理框图进行对照，可以看出该原理图符合设计需求。接着对工程进行约束，由于本设计为组合电路，因此无需进行时序约束，只需进行引脚约束。对于输入 aIn 和 bIn，我采用开关约束，其中，SW7-SW4 为 aIn，SW3-SW0 为 bIn，而对于输出 out，我采用指示灯约束，LD3-LD0 为 out，三者均是依次从高位到低位，接口类型均为 LVCMOS33。最后将工程文件上传到 Nexys Video 开发板上，工程文件的建立完成。

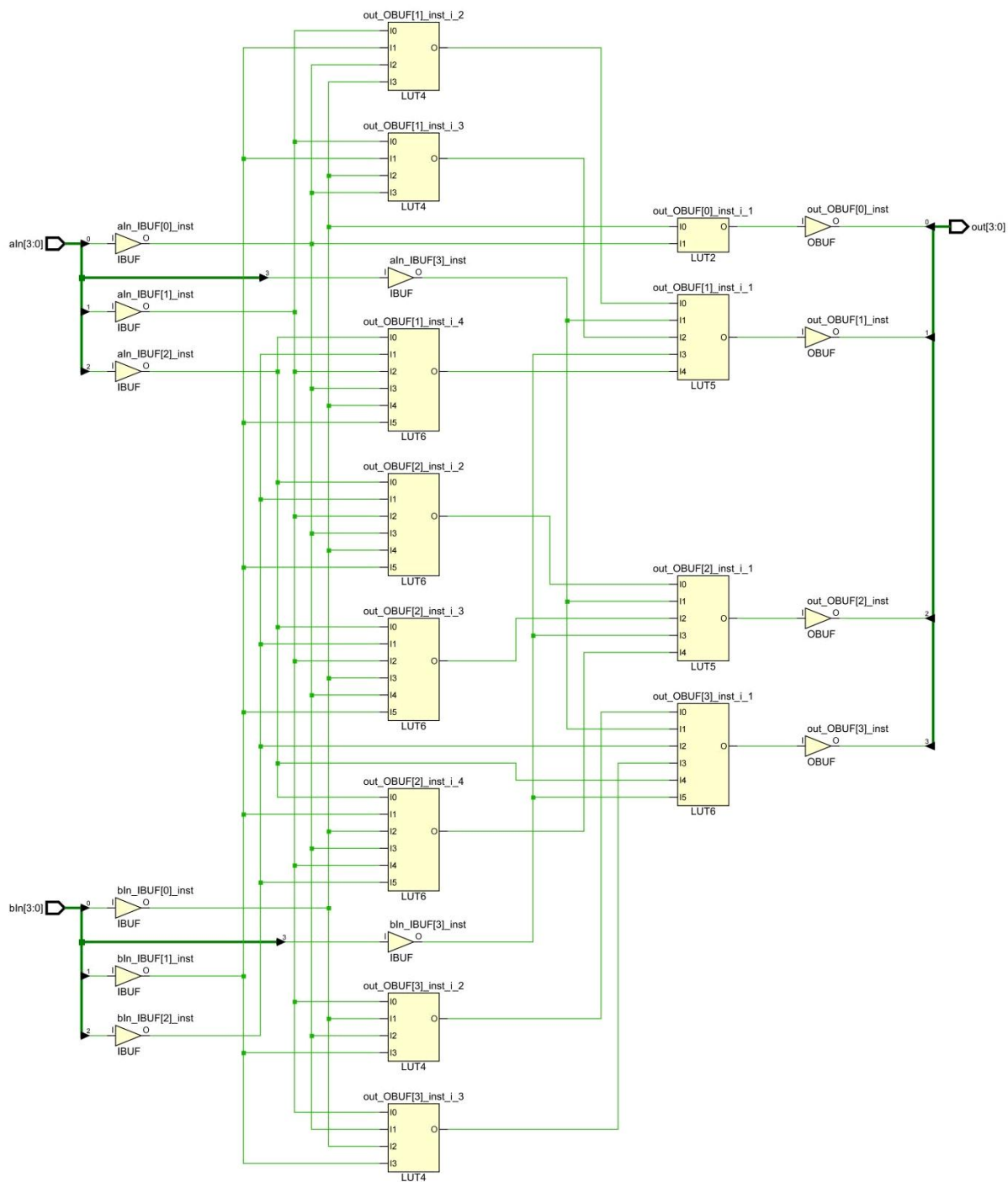


图 7 任务一原理图

2、任务二 模式比较器

模式比较器的 ModelSim 仿真波形如图 8 所示。如图，当 $m=0$ 时，输出为输入的最大值，当 $m=1$ 时，输出为输入的最小值，输出的取值与该模块需要实现的功能一致，该模块程序编写正确，可以正常运行。

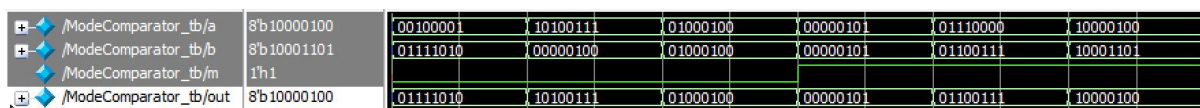


图 8 模式比较器的仿真波形

八、开发板测试

为检验任务一是否能够在开发板上实现，将其工程文件上传到 Nexys Video 开发板上进行测试。先令 $aIn=4'b0011$, $bIn=4'b1100$, 需将开关 SW5、SW4、SW3、SW2 闭合, SW7、SW6、SW1、SW0 断开。此时指示灯 LD3、LD0 亮, LD2、LD1 灭, 如图 9 所示, $out=4'b1001=|a-b|$, 符合预期; 再令 $aIn=4'b1010$, $bIn=4'b0100$, 需将开关 SW7、SW5、SW2 闭合, SW6、SW4、SW3、SW1、SW0 断开。此时指示灯 LD2、LD1 亮, LD3、LD0 灭, 如图 10 所示, $out=4'b0110=|a-b|$, 同样符合预期。由此, 功能可以正常实现, 开发板通过测试。

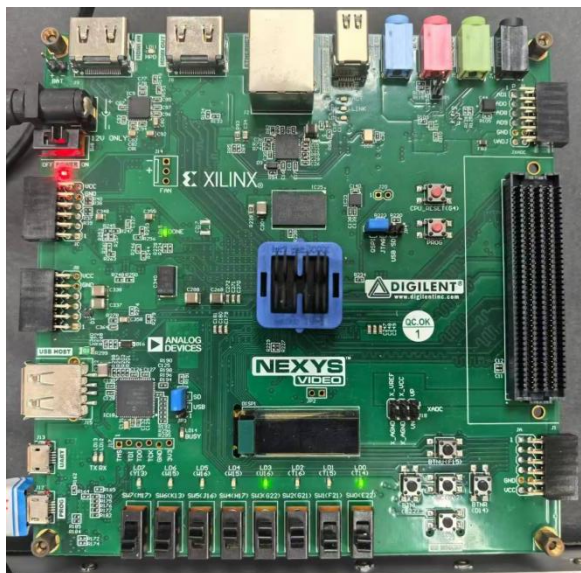


图 9 $aIn=4'b0011$, $bIn=4'b1100$

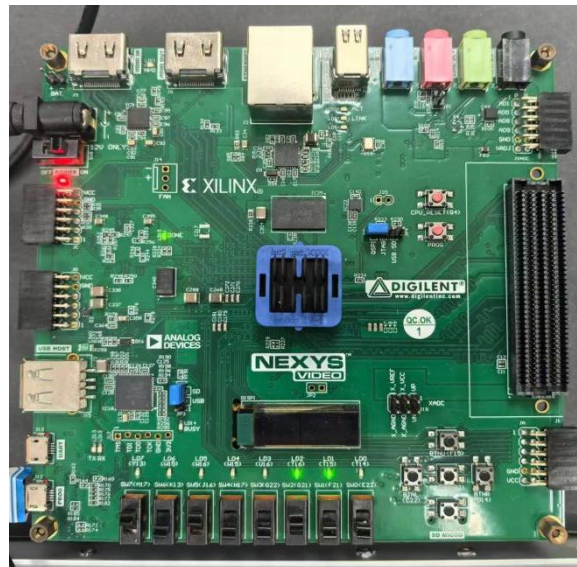


图 10 $aIn=4'b1010$, $bIn=4'b0100$

九、总结、心得

本次实验我使用 ModelSim 和 Vivado 软件, 应用 Verilog HDL 语言描述, 设计出了两数之差的绝对值电路和模式比较器。在本次实验中, 我在设计代码的过程中学习到了 Verilog HDL 语言的多种语句结构, 进一步了解了加法器电路等基本的数字电路, 也更加熟悉了这两个软件的使用流程。在本次实验中, 我发现我在完成实验 1 和实验 2 时对 ModelSim 仿真图的信息识别能力和对 Vivado 软件的综合后的原理图的重重视程度有所欠缺, 这告诉了我在今后的实验中要多重视细节的理解和体会, 而不是仅仅局限于按照书本中的步骤做出成果。

十、思考题

(1)求两数之差的绝对值电路, 若输入为有符号数(补码), 该怎样设计? 请画出原理框图并作简要说明。

输入补码时, 可以使输出为无符号数, 此时需要让 $addr$ 在 $aIn > bIn > 0$ 、 $aIn > 0 > bIn$ 、 $aIn < bIn < 0$ 三种情况下取 0, 其他情况下取 1。

假设输入为 4 位有符号二进制数, 则 $addr = aIn[3] \& (\sim(bIn[3])) \mid aIn[3] \& bIn[3] \& (\sim agb) \mid (\sim aIn[3]) \& (\sim bIn[3]) \& (\sim agb)$ 。原理框图如图 11 所示。

(2)能否用加法器实现比较器, 若能, 请画出原理框图。

能。因为加法器可以计算两数之差, 故可以使用输出值和溢出位来比较两数大小。若 $a > b$, 则溢出位为 1, 输出位不全为 0; 若 $a = b$, 则溢出位为 0, 输出位全为 0; 若 $a < b$, 则溢出位为 1, 输出位不全为 0。

因此, 利用输出位和溢出位, 结合必要的门电路, 即可得到 agb 、 aeb 和 alb , 实现比较器, 原理框图如图 12 所示。

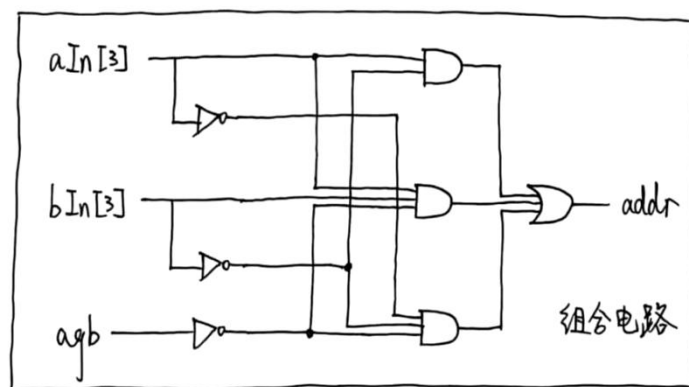
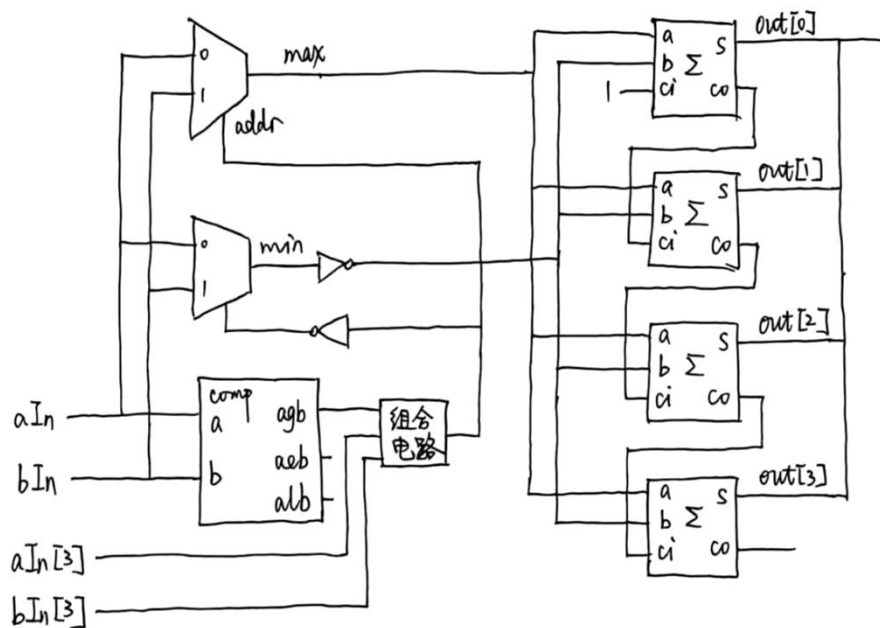


图 11 思考题 1 原理框图

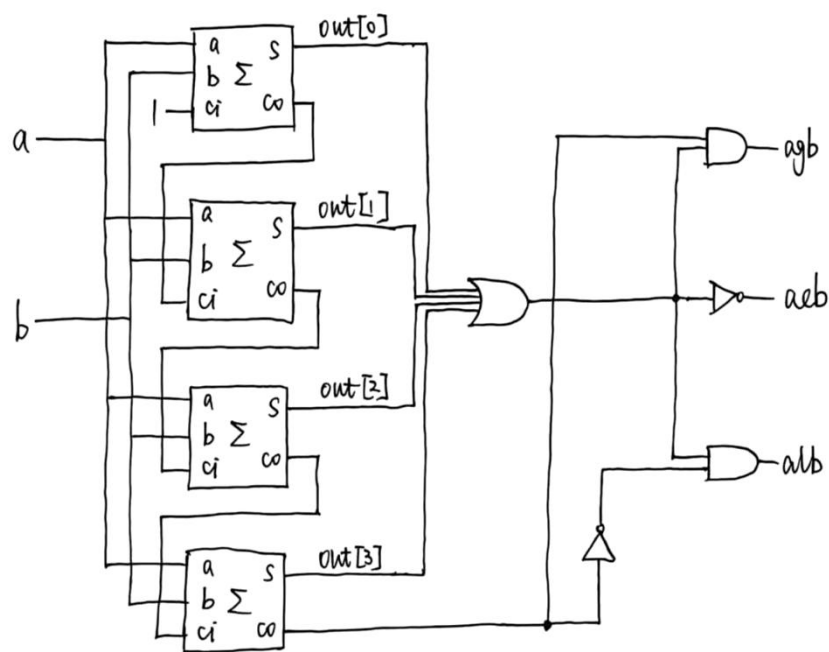


图 12 思考题 2 原理框图

(3)调用模块时怎样进行参数传递？若模块有多个参数，模块实例的格式如何？

在 Verilog HDL 语言中，参数传递通过#(参数列表)进行参数传递。传递方式有两种：一种是按顺序传递，程序语言为#(值 1, 值 2, …)，值的顺序需要对应模块定义中的参数顺序；另一种是按名称传递，程序语言为#(.参数名 1(值 1), .参数名 2(值 2), …)，这种方式与顺序无关。